SMART INDIA HACKATHON 2026

AI-POWERED CRIMINAL NETWORK ANALYSIS SYSTEM

A Software-Based Platform Proposal for Ministry of Home Affairs (MHA)

Category:	Software Edition (100% Code-based)
Problem Statement ID:	SIH26189
Focus Domain:	AI/ML, Graph Analytics, Cyber Investigations
Primary Tech Stack:	FastAPI, React, Neo4j, Vis.js, NLP (spaCy)

Prepared by: NIT Delhi Hackathon Team
AI Mentor: Antigravity Assistant
Date: August 30, 2026

1. Executive Summary & Concept

Modern organized crime syndicates operate across jurisdictions, masking their actions through a complex web of throwaway phones, decoy bank accounts, and localized relationships. While law enforcement agencies (LEAs) collect copious amounts of raw data—such as First Information Reports (FIRs), Call Detail Records (CDRs), and Bank Statements—these records are stored in siloed, unstructured formats. Investigators are forced to manually correlate these scattered files on paper charts or disjointed Excel files.

The **AI-Powered Criminal Network Analysis System** automatically reads these heterogeneous files, extracts names, aliases, locations, phone numbers, and financial details, and fuses them into a central **Graph Database**. It then runs mathematical graph algorithms to reveal key organizational patterns.

Core Ingestion-to-Visualization Pipeline:

- **1. File Upload:** The investigator uploads raw, multi-format files (PDF case reports, CSV phone logs, Excel bank sheets).
- **2. Multi-lingual AI Text Parser (NLP):** An NLP pipeline identifies entities. If reports are in regional Indian languages (e.g., Hindi), a translation bridge translates/transliterates them to standardized English.
- **3. Entity Resolution:** The system resolves duplicate entities (e.g., if a suspect 'Rajesh' uses the same phone number as 'Rajesh Kumar' in a different report, they are merged into one node).
- **4. Graph Relational Database (Neo4j):** Nodes and relationships (e.g., 'A called B', 'B transferred funds to C') are stored in a graph network.
- **5. Graph Analytics:** Algorithms compute PageRank/Centrality scores to find 'hubs' (ring leaders) and trace Dijkstra's Shortest Path to link remote suspects.
- **6. Interactive Dashboard (Vis.js):** The network is rendered as an interactive, draggable visual interface with search, filters, and timeline playbacks.

2. Under the Hood: Detailed System Working

To understand how the software processes data dynamically, we trace the step-by-step processing of a typical case investigation:

Phase A: Data Parsing & NLP Extraction

When an unstructured narrative file like an FIR is uploaded, a Python backend reads the text block. It applies **Named Entity Recognition (NER)** (using spaCy or a local transformer model) to tag entities:

Input Text: "The victim Pooja was tricked into sending Rs. 25,000 to Bank Account 9998887776 belonging to Rajesh. Rajesh contacted her from phone +91 90000 11111."
Output Entities:
• **Pooja** → [Label: VICTIM]
• **Rajesh** → [Label: SUSPECT]
• **9998887776** → [Label: BANK_ACCOUNT] (extracted via Regex)
• **+91 90000 11111** → [Label: PHONE] (extracted via Regex)

Phase B: Ingesting Tabular CSV/Excel Records

Simultaneously, Call Detail Records (CDRs) and financial statements are uploaded. The system reads them as dataframes and creates relational links:

CDR Edge: +91 90000 11111 —[CALLED (Date: 2026-08-25, Dur: 120s)]→ +91 88888 22222
Bank Edge: Account 9998887776 —[TRANSFERRED (Amount: Rs.20000, Date: 2026-08-26)]→ Account 5554443332

Phase C: Entity Resolution & Graph Building

If a previous report contained a suspect node for 'Rajesh Kumar' associated with phone number '+91 90000 11111', the engine recognizes the match and merges the new 'Rajesh' suspect node with the existing node. The system connects the records:

$$\text{Pooja (Victim)} \xrightarrow{\text{Scammed}} \text{Rajesh (Suspect)} \xrightarrow{\text{Uses Phone}} \text{+91 90000 11111} \xrightarrow{\text{Calls}} \text{+91 88888 22222}$$

Phase D: Graph Analytics APIs

Once loaded into the memory graph, the backend runs calculations:
1. **PageRank Centrality:** Scores nodes based on link numbers and connected node values, highlighting gang leaders.
2. **Shortest Path Traversal:** Traces Dijkstra's algorithm to link two distant nodes (e.g., linking a victim to a hidden beneficiary).

3. Market Comparison & Component Stack

This project bridges the gap between manual police workflows and multi-million dollar software suites. A breakdown of how this solution compares to the current methods:

Criteria	Police Method (Current)	Enterprise (Palantir/i2)	Our Proposed System
Input Ingestion	Manual transcription of text logs and statements.	Ingests clean SQL databases and pre-processed files.	**Automated AI parsing** of raw unstructured PDFs/Texts.
Data Fusion	Siloed Excel sheets and paper archives.	Custom DB connectors, high setup time.	**Fuses FIR narratives + CDRs + Transactions** automatically.
Localization	Bilingual investigators do manual reading.	No regional Indian language parser.	**Indian Vernacular**: Built-in translation layers.
Cost & Setup	Free, but highly time-consuming and error-prone.	Heavy licensing, steep learning curve.	**Open-source**, browser-based, zero setup fallback.

System Component Tech Stack:

Layer	Technologies	Role
Frontend UI	React / Tailwind CSS	Responsive web interface for file uploads & search.
Graph Rendering	Vis.js	Physics-simulated canvas displaying interactive nodes.
Backend API	FastAPI (Python)	Exposes endpoints for NLP parsing and graph analysis.
NLP AI Engine	spaCy / HuggingFace Transformers	Entity extraction (NER) and name translation.
Graph Core	Neo4j / NetworkX (fallback)	Performs PageRank, Shortest Path, and clustering.

4. Implementation Roadmap & Strategy

To deliver a highly polished, working prototype in the 36-hour hackathon, we follow this developmental plan:

Hours	Target Module	Deliverables
0 - 6	Scaffolding	Initialize FastAPI backend, set up React dashboard, install Vis.js core.
6 - 12	NLP Ingestion	Write text readers & regex matchers. Add translation model for regional texts.
12 - 18	Graph Modeller	Integrate NetworkX/Neo4j database logic to save nodes and relationships.
18 - 24	Analytics APIs	Deploy PageRank (leaders) & Dijkstra (shortest path) endpoints.
24 - 30	Dashboard UI	Connect graph visualization to APIs. Implement node dragging & timeline slider.
30 - 36	Polishing & Demo	Ingest mock files (sample case), test validation runs, create auto-run script.

Judges' Evaluation High-Impact Highlights:

- **Zero-Setup Fallback DB:** If Neo4j is offline, the backend seamlessly switches to an in-memory Graph structure, guaranteeing a crash-free demo.
- **Temporal Timeline Slider:** A visual slider to play back how the gang's calls and transactions occurred week-by-week.
- **Regional Language Parsing:** Demonstrating real-world utility by extracting data from local-language reports.

Conclusion: This proposal matches the Ministry of Home Affairs' guidelines for SIH26189, combining sophisticated backend intelligence with a zero-setup browser interface, making it the ideal hackathon candidate.